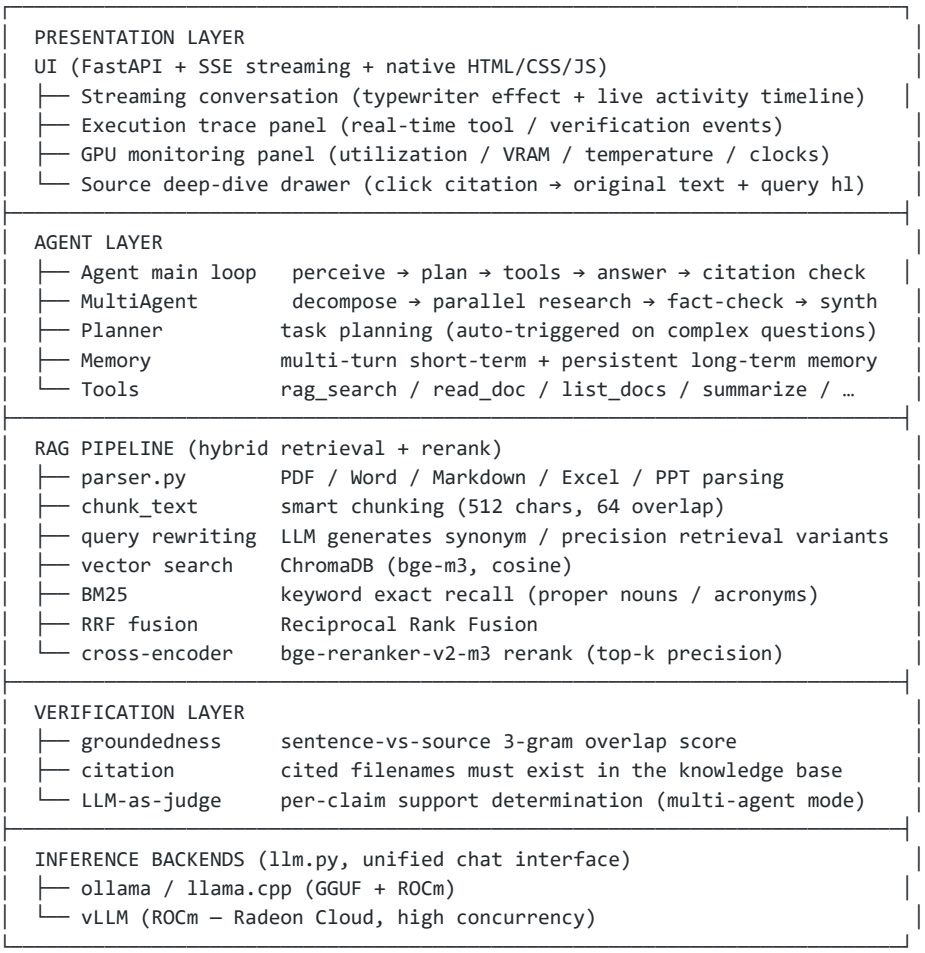


Project Specification — Private RAG Agent

A Fully Offline, Local Private Multi-Agent RAG Assistant Optimized for AMD Radeon GPUs

Field	Value
Competition	2026 AMD AI DevMaster Hackathon — Track 2: Development & Local Deployment of Private AI Agents
Track Focus	Local deployment of agentic private AI; 40% of score from AMD Radeon GPU / ROCm optimization
Project Name	Private RAG Agent (`private-rag-agent`)

Field	Value
Core Principle	Data never leaves the machine. Chunking, embedding, retrieval, reasoning and verification all run locally. No cloud API dependency at runtime.
Team	MENG Yuxuan
Demo Video	[demo_video.md](../submissions/track2-private-rag-agent/demo_video.md) — Bilibili / YouTube (recording)
Submission Deadline	2026-08-06 23:59 (UTC+8)
Competition	2026 AMD AI DevMaster Hackathon — Track 2: Development & Local Deployment of Private AI Agents
Track Focus	Local deployment of agentic private AI; 40% of score from AMD Radeon GPU / ROCm optimization
Project Name	Private RAG Agent (`private-rag-agent`)
Core Principle	Data never leaves the machine. Chunking, embedding, retrieval, reasoning and verification all run locally. No cloud API dependency at runtime.
Team	MENG Yuxuan
Demo Video	[demo_video.md](../submissions/track2-private-rag-agent/demo_video.md) — Bilibili / YouTube (recording)

Competition	2026 AMD AI DevMaster Hackathon — Track 2: Development & Local Deployment of Private AI Agents
Submission Deadline	2026-08-06 23:59 (UTC+8)
Competition	2026 AMD AI DevMaster Hackathon — Track 2: Development & Local Deployment of Private AI Agents
Track Focus	Local deployment of agentic private AI; 40% of score from AMD Radeon GPU / ROCm optimization
Project Name	Private RAG Agent (`private-rag-agent`)
Core Principle	Data never leaves the machine. Chunking, embedding, retrieval, reasoning and verification all run locally. No cloud API dependency at runtime.
Team	MENG Yuxuan
Demo Video	[demo_video.md](../submissions/track2-private-rag-agent/demo_video.md) — Bilibili / YouTube (recording)
Submission Deadline	2026-08-06 23:59 (UTC+8)

Track Focus	Local deployment of agentic private AI; 40% of score from AMD Radeon GPU / ROCm optimization
Project Name	Private RAG Agent (`private-rag-agent`)
Core Principle	Data never leaves the machine. Chunking, embedding, retrieval, reasoning and verification all run locally. No cloud API dependency at runtime.
Team	MENG Yuxuan
Demo Video	[demo_video.md](../submissions/track2-private-rag-agent/demo_video.md) — Bilibili / YouTube (recording)
Submission Deadline	2026-08-06 23:59 (UTC+8)
Project Name	Private RAG Agent (`private-rag-agent`)
Core Principle	Data never leaves the machine. Chunking, embedding, retrieval, reasoning and verification all run locally. No cloud API dependency at runtime.
Team	MENG Yuxuan
Demo Video	[demo_video.md](../submissions/track2-private-rag-agent/demo_video.md) — Bilibili / YouTube (recording)
Submission Deadline	2026-08-06 23:59 (UTC+8)

Core Principle	Data never leaves the machine. Chunking, embedding, retrieval, reasoning and verification all run locally. No cloud API dependency at runtime.
Team	MENG Yuxuan
Demo Video	[demo_video.md](../submissions/track2-private-rag-agent/demo_video.md) — Bilibili / YouTube (recording)
Submission Deadline	2026-08-06 23:59 (UTC+8)
Team	MENG Yuxuan
Demo Video	[demo_video.md](../submissions/track2-private-rag-agent/demo_video.md) — Bilibili / YouTube (recording)
Submission Deadline	2026-08-06 23:59 (UTC+8)
Demo Video	[demo_video.md](../submissions/track2-private-rag-agent/demo_video.md) — Bilibili / YouTube (recording)
Submission Deadline	2026-08-06 23:59 (UTC+8)
Submission Deadline	2026-08-06 23:59 (UTC+8)

Table of Contents

---

1. Executive Summary

Private RAG Agent is an all-offline, locally deployed multi-agent Retrieval-Augmented Generation (RAG) system. Given a complex question, it decomposes the question into independent sub-questions, dispatches parallel research agents that retrieve from a private knowledge base and write sub-reports, verifies every factual statement against the retrieved source text, and finally composes a single answer using only the content that passed verification. Every sentence of the final answer is auditable: each citation links back to the exact source passage, and each sentence is labeled supported, partially supported, or unsupported with a quantitative grounding score.

The system is architected around AMD Radeon GPUs and the ROCm software stack from day one. It supports three interchangeable inference backends (Ollama/ROCm, llama.cpp/HIP, vLLM/ROCm), runs quantized GGUF models with full-layer GPU offload, and exploits a researcher-pool parallelism strategy to saturate GPU throughput — the key lever behind the 5–10x wall-clock speedup projected versus CPU inference.

---

2. Application Scenarios

The design targets organizations and individuals who cannot or will not send their data to third-party clouds. Representative scenarios:

2.1 Private / Proprietary Knowledge Bases

Companies maintain confidential documentation — product technical manuals, internal white papers, compliance rules, project planning documents. The assistant indexes these documents locally and answers questions grounded strictly in the ingested corpus, with citations that can be traced back to the original text. All parsing, embedding, retrieval, generation and verification are executed on-premises; no document content is ever transmitted to an external service.

2.2 Offline / Air-Gapped Environments

Defense, energy and industrial-control sites frequently operate on isolated networks with no internet access. Because every component — model weights, vector store, reranker, and LLM — is local, the system runs with the network cable disconnected. The only prerequisite is a machine with sufficient CPU/GPU resources.

2.3 Enterprises with Data-Privacy and Compliance Requirements

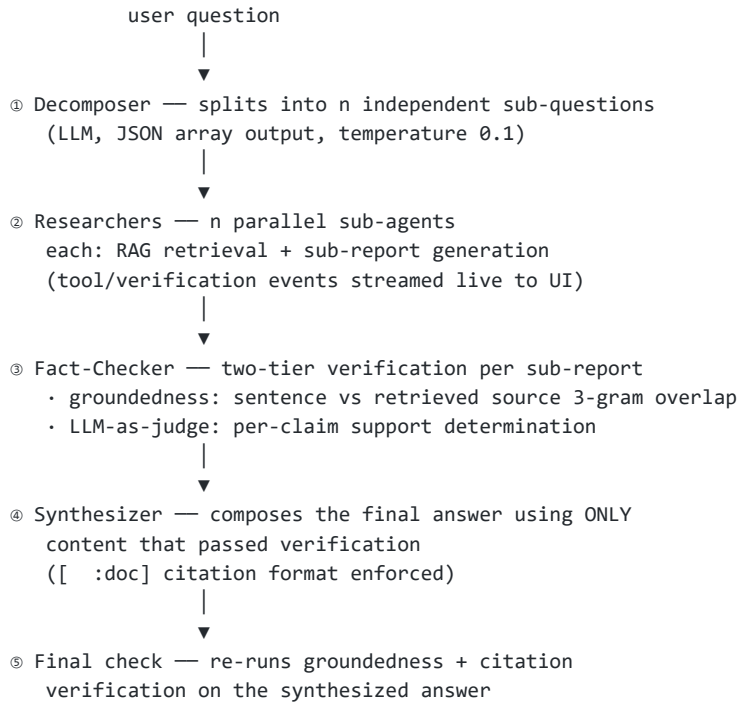
GDPR, HIPAA, PIPL and other regimes impose strict controls on personal and sensitive data. This system provides a deterministic answer pipeline where the provenance of each statement is machine-checkable, which supports compliance audits ("which source supports this claim?") in a way that closed-box cloud assistants cannot.

2.4 Local AI Assistants on AMD Workstations

For individual engineers and analysts, the system runs as a single-machine assistant on an AMD Radeon workstation (recommended 16GB+ VRAM, e.g. Radeon RX 7900 XTX / W7900). A single browser UI provides document ingestion, hybrid search, GPU monitoring, and a fully transparent reasoning trail, with no per-token cloud billing.

3. System Architecture

3.1 End-to-End Architecture



3.2 Multi-Agent Collaboration Pipeline

```
python main.py ingest data/docs/   # batch-import documents
python main.py ui                  # http://localhost:7860
```

Parallelism rationale (AMD-specific). On a local GPU, multiple inference requests queue for the same device. Serially executing a multi-step research task multiplies wall-clock latency; running the `n` researchers concurrently cuts the wall-clock time of multi-step research tasks substantially. Meanwhile the `n` sub-agents share a single copy of the embedding and reranker models (module-level singletons), so VRAM overhead grows by only one copy, not `n`.

3.3 Component Responsibilities

Component	Responsibility		
`agent/agent.py`	Single-agent main loop: intent gating, planning, streaming tool execution, citation verification		
`agent/multi_agent.py`	Orchestration: decompose → parallel research → fact-check → synthesize, with a rich SSE event stream		

Component	Responsibility		
`agent/rag.py`	Hybrid retrieval: query rewrite + vector ∪ BM25 → RRF → cross-encoder rerank		
`agent/verify.py`	Deterministic groundedness (n-gram) and citation-existence verification		
`agent/llm.py`	Unified inference backend abstraction (ollama / llama_cpp / vLLM-openai-compat)		
`agent/parser.py`	Multi-format document parsing and smart chunking		
`agent/planner.py`	Task planning; `agent/memory.py` short- and long-term memory		
`ui/server.py`	FastAPI + SSE backend; document ingest; session management; GPU telemetry		
`benchmarks/bench_amd.py`	Automatic AMD/ROCm detection and tokens/s benchmark harness		
Role	Model	Size / Type	Rationale
LLM	Qwen3-8B (`qwen3:8b`)	8B decoder-only	Strong function calling, multilingual (incl. Chinese), fits 16GB-class Radeon GPUs when quantized
Embedding	BAAI/bge-m3	1024-dim multilingual embedding	High-quality Chinese/English retrieval; used for both ingestion and query encoding
Reranker	BAAI/bge-reranker-v2-m3	cross-encoder (512 tokens)	Multilingual cross-encoder; substantially improves top-k precision over bi-encoder scores

Component	Responsibility		
Backend	Runtime stack	Target environment	
Ollama	Ollama with automatic ROCm device backend on Linux (`ollama ps` confirms 100% GPU)	Simplest local setup; Windows/macOS/Linux development	
llama.cpp (HIP)	GGUF + `CMAKE_ARGS="-DGGML_HIP=ON"`, `n_gpu_layers=-1` full GPU offload	Single-machine, single-GPU, maximum control over quantization	
vLLM (ROCm)	`rocm/vllm` container; OpenAI-compatible API; high concurrency	Radeon Cloud and multi-tenant/high-concurrency GPU environments	
Quantization	Approx. VRAM	Speed	Recommendation
`Q8_0`	~8.2 GB	Fast	Quality-first deployments
`Q4_K_M`	~4.6 GB	Fastest	Balanced default recommendation
`agent/agent.py`	Single-agent main loop: intent gating, planning, streaming tool execution, citation verification		
`agent/multi_agent.py`	Orchestration: decompose → parallel research → fact-check → synthesize, with a rich SSE event stream		
`agent/rag.py`	Hybrid retrieval: query rewrite + vector ∪ BM25 → RRF → cross-encoder rerank		
`agent/verify.py`	Deterministic groundedness (n-gram) and citation-existence verification		
`agent/llm.py`	Unified inference backend abstraction (ollama / llama_cpp / vLLM-openai-compat)		

`agent/agent.py`	Single-agent main loop: intent gating, planning, streaming tool execution, citation verification		
`agent/parser.py`	Multi-format document parsing and smart chunking		
`agent/planner.py`	Task planning; `agent/memory.py` short- and long-term memory		
`ui/server.py`	FastAPI + SSE backend; document ingest; session management; GPU telemetry		
`benchmarks/bench_amd.py`	Automatic AMD/ROCm detection and tokens/s benchmark harness		
Role	Model	Size / Type	Rationale
LLM	Qwen3-8B (`qwen3:8b`)	8B decoder-only	Strong function calling, multilingual (incl. Chinese), fits 16GB-class Radeon GPUs when quantized
Embedding	BAAI/bge-m3	1024-dim multilingual embedding	High-quality Chinese/English retrieval; used for both ingestion and query encoding
Reranker	BAAI/bge-reranker-v2-m3	cross-encoder (512 tokens)	Multilingual cross-encoder; substantially improves top-k precision over bi-encoder scores
Backend	Runtime stack	Target environment	
Ollama	Ollama with automatic ROCm device backend on Linux (`ollama ps` confirms 100% GPU)	Simplest local setup; Windows/macOS/Linux development	

`agent/agent.py`	Single-agent main loop: intent gating, planning, streaming tool execution, citation verification		
llama.cpp (HIP)	GGUF + `CMAKE_ARGS="-DGGML_HIP=ON"`, `n_gpu_layers=-1` full GPU offload	Single-machine, single-GPU, maximum control over quantization	
vLLM (ROCm)	`rocm/vllm` container; OpenAI-compatible API; high concurrency	Radeon Cloud and multi-tenant/high-concurrency GPU environments	
Quantization	Approx. VRAM	Speed	Recommendation
`Q8_0`	~8.2 GB	Fast	Quality-first deployments
`Q4_K_M`	~4.6 GB	Fastest	Balanced default recommendation
`agent/agent.py`	Single-agent main loop: intent gating, planning, streaming tool execution, citation verification		
`agent/multi_agent.py`	Orchestration: decompose → parallel research → fact-check → synthesize, with a rich SSE event stream		
`agent/rag.py`	Hybrid retrieval: query rewrite + vector ∪ BM25 → RRF → cross-encoder rerank		
`agent/verify.py`	Deterministic groundedness (n-gram) and citation-existence verification		
`agent/llm.py`	Unified inference backend abstraction (ollama / llama_cpp / vLLM-openai-compat)		
`agent/parser.py`	Multi-format document parsing and smart chunking		

`agent/agent.py`	Single-agent main loop: intent gating, planning, streaming tool execution, citation verification		
`agent/planner.py`	Task planning; `agent/memory.py` short- and long-term memory		
`ui/server.py`	FastAPI + SSE backend; document ingest; session management; GPU telemetry		
`benchmarks/bench_amd.py`	Automatic AMD/ROCm detection and tokens/s benchmark harness		
Role	Model	Size / Type	Rationale
LLM	Qwen3-8B (`qwen3:8b`)	8B decoder-only	Strong function calling, multilingual (incl. Chinese), fits 16GB-class Radeon GPUs when quantized
Embedding	BAAI/bge-m3	1024-dim multilingual embedding	High-quality Chinese/English retrieval; used for both ingestion and query encoding
Reranker	BAAI/bge-reranker-v2-m3	cross-encoder (512 tokens)	Multilingual cross-encoder; substantially improves top-k precision over bi-encoder scores
Backend	Runtime stack	Target environment	
Ollama	Ollama with automatic ROCm device backend on Linux (`ollama ps` confirms 100% GPU)	Simplest local setup; Windows/macOS/Linux development	
llama.cpp (HIP)	GGUF + `CMAKE_ARGS="-DGGML_HIP=ON"`, `n_gpu_layers=-1` full GPU offload	Single-machine, single-GPU, maximum control over quantization	

`agent/agent.py`	Single-agent main loop: intent gating, planning, streaming tool execution, citation verification		
vLLM (ROCm)	`rocm/vllm` container; OpenAI-compatible API; high concurrency	Radeon Cloud and multi-tenant/high-concurrency GPU environments	
Quantization	Approx. VRAM	Speed	Recommendation
`Q8_0`	~8.2 GB	Fast	Quality-first deployments
`Q4_K_M`	~4.6 GB	Fastest	Balanced default recommendation
`agent/multi_agent.py`	Orchestration: decompose → parallel research → fact-check → synthesize, with a rich SSE event stream		
`agent/rag.py`	Hybrid retrieval: query rewrite + vector ∪ BM25 → RRF → cross-encoder rerank		
`agent/verify.py`	Deterministic groundedness (n-gram) and citation-existence verification		
`agent/llm.py`	Unified inference backend abstraction (ollama / llama_cpp / vLLM-openai-compat)		
`agent/parser.py`	Multi-format document parsing and smart chunking		
`agent/planner.py`	Task planning; `agent/memory.py` short- and long-term memory		
`ui/server.py`	FastAPI + SSE backend; document ingest; session management; GPU telemetry		

`agent/multi_agent.py`	Orchestration: decompose → parallel research → fact-check → synthesize, with a rich SSE event stream		
`benchmarks/bench_amd.py`	Automatic AMD/ROCm detection and tokens/s benchmark harness		
Role	Model	Size / Type	Rationale
LLM	Qwen3-8B (`qwen3:8b`)	8B decoder-only	Strong function calling, multilingual (incl. Chinese), fits 16GB-class Radeon GPUs when quantized
Embedding	BAAI/bge-m3	1024-dim multilingual embedding	High-quality Chinese/English retrieval; used for both ingestion and query encoding
Reranker	BAAI/bge-reranker-v2-m3	cross-encoder (512 tokens)	Multilingual cross-encoder; substantially improves top-k precision over bi-encoder scores
Backend	Runtime stack	Target environment	
Ollama	Ollama with automatic ROCm device backend on Linux (`ollama ps` confirms 100% GPU)	Simplest local setup; Windows/macOS/Linux development	
llama.cpp (HIP)	GGUF + `CMAKE_ARGS="-DGGML_HIP=ON"`, `n_gpu_layers=-1` full GPU offload	Single-machine, single-GPU, maximum control over quantization	
vLLM (ROCm)	`rocm/vllm` container; OpenAI-compatible API; high concurrency	Radeon Cloud and multi-tenant/high-concurrency GPU environments	
Quantization	Approx. VRAM	Speed	Recommendation
`Q8_0`	~8.2 GB	Fast	Quality-first deployments
`Q4_K_M`	~4.6 GB	Fastest	Balanced default recommendation

`agent/rag.py`	Hybrid retrieval: query rewrite + vector ∪ BM25 → RRF → cross-encoder rerank		
`agent/verify.py`	Deterministic groundedness (n-gram) and citation-existence verification		
`agent/llm.py`	Unified inference backend abstraction (ollama / llama_cpp / vLLM-openai-compat)		
`agent/parser.py`	Multi-format document parsing and smart chunking		
`agent/planner.py`	Task planning; `agent/memory.py` short- and long-term memory		
`ui/server.py`	FastAPI + SSE backend; document ingest; session management; GPU telemetry		
`benchmarks/bench_amd.py`	Automatic AMD/ROCm detection and tokens/s benchmark harness		
Role	Model	Size / Type	Rationale
LLM	Qwen3-8B (`qwen3:8b`)	8B decoder-only	Strong function calling, multilingual (incl. Chinese), fits 16GB-class Radeon GPUs when quantized
Embedding	BAAI/bge-m3	1024-dim multilingual embedding	High-quality Chinese/English retrieval; used for both ingestion and query encoding
Reranker	BAAI/bge-reranker-v2-m3	cross-encoder (512 tokens)	Multilingual cross-encoder; substantially improves top-k precision over bi-encoder scores
Backend	Runtime stack	Target environment	

`agent/rag.py`	Hybrid retrieval: query rewrite + vector ∪ BM25 → RRF → cross-encoder rerank		
Ollama	Ollama with automatic ROCm device backend on Linux (`ollama ps` confirms 100% GPU)	Simplest local setup; Windows/macOS/Linux development	
llama.cpp (HIP)	GGUF + `CMAKE_ARGS="-DGGML_HIP=ON"`, `n_gpu_layers=-1` full GPU offload	Single-machine, single-GPU, maximum control over quantization	
vLLM (ROCm)	`rocm/vllm` container; OpenAI-compatible API; high concurrency	Radeon Cloud and multi-tenant/high-concurrency GPU environments	
Quantization	Approx. VRAM	Speed	Recommendation
`Q8_0`	~8.2 GB	Fast	Quality-first deployments
`Q4_K_M`	~4.6 GB	Fastest	Balanced default recommendation
`agent/verify.py`	Deterministic groundedness (n-gram) and citation-existence verification		
`agent/llm.py`	Unified inference backend abstraction (ollama / llama_cpp / vLLM-openai-compat)		
`agent/parser.py`	Multi-format document parsing and smart chunking		
`agent/planner.py`	Task planning; `agent/memory.py` short- and long-term memory		
`ui/server.py`	FastAPI + SSE backend; document ingest; session management; GPU telemetry		
`benchmarks/bench_amd.py`	Automatic AMD/ROCm detection and tokens/s benchmark harness		

`agent/verify.py`	Deterministic groundedness (n-gram) and citation-existence verification		
Role	Model	Size / Type	Rationale
LLM	Qwen3-8B (`qwen3:8b`)	8B decoder-only	Strong function calling, multilingual (incl. Chinese), fits 16GB-class Radeon GPUs when quantized
Embedding	BAAI/bge-m3	1024-dim multilingual embedding	High-quality Chinese/English retrieval; used for both ingestion and query encoding
Reranker	BAAI/bge-reranker-v2-m3	cross-encoder (512 tokens)	Multilingual cross-encoder; substantially improves top-k precision over bi-encoder scores
Backend	Runtime stack	Target environment	
Ollama	Ollama with automatic ROCm device backend on Linux (`ollama ps` confirms 100% GPU)	Simplest local setup; Windows/macOS/Linux development	
llama.cpp (HIP)	GGUF + `CMAKE_ARGS="-DGGML_HIP=ON"`, `n_gpu_layers=-1` full GPU offload	Single-machine, single-GPU, maximum control over quantization	
vLLM (ROCm)	`rocm/vllm` container; OpenAI-compatible API; high concurrency	Radeon Cloud and multi-tenant/high-concurrency GPU environments	
Quantization	Approx. VRAM	Speed	Recommendation
`Q8_0`	~8.2 GB	Fast	Quality-first deployments
`Q4_K_M`	~4.6 GB	Fastest	Balanced default recommendation

`agent/llm.py`	Unified inference backend abstraction (ollama / llama_cpp / vLLM-openai-compat)		
`agent/parser.py`	Multi-format document parsing and smart chunking		
`agent/planner.py`	Task planning; `agent/memory.py` short- and long-term memory		
`ui/server.py`	FastAPI + SSE backend; document ingest; session management; GPU telemetry		
`benchmarks/bench_amd.py`	Automatic AMD/ROCm detection and tokens/s benchmark harness		
Role	Model	Size / Type	Rationale
LLM	Qwen3-8B (`qwen3:8b`)	8B decoder-only	Strong function calling, multilingual (incl. Chinese), fits 16GB-class Radeon GPUs when quantized
Embedding	BAAI/bge-m3	1024-dim multilingual embedding	High-quality Chinese/English retrieval; used for both ingestion and query encoding
Reranker	BAAI/bge-reranker-v2-m3	cross-encoder (512 tokens)	Multilingual cross-encoder; substantially improves top-k precision over bi-encoder scores
Backend	Runtime stack	Target environment	
Ollama	Ollama with automatic ROCm device backend on Linux (`ollama ps` confirms 100% GPU)	Simplest local setup; Windows/macOS/Linux development	
llama.cpp (HIP)	GGUF + `CMAKE_ARGS="-DGGML_HIP=ON"`, `n_gpu_layers=-1` full GPU offload	Single-machine, single-GPU, maximum control over quantization	

`agent/llm.py`	Unified inference backend abstraction (ollama / llama_cpp / vLLM-openai-compat)		
vLLM (ROCm)	`rocm/vllm` container; OpenAI-compatible API; high concurrency	Radeon Cloud and multi-tenant/high-concurrency GPU environments	
Quantization	Approx. VRAM	Speed	Recommendation
`Q8_0`	~8.2 GB	Fast	Quality-first deployments
`Q4_K_M`	~4.6 GB	Fastest	Balanced default recommendation
`agent/parser.py`	Multi-format document parsing and smart chunking		
`agent/planner.py`	Task planning; `agent/memory.py` short- and long-term memory		
`ui/server.py`	FastAPI + SSE backend; document ingest; session management; GPU telemetry		
`benchmarks/bench_amd.py`	Automatic AMD/ROCm detection and tokens/s benchmark harness		
Role	Model	Size / Type	Rationale
LLM	Qwen3-8B (`qwen3:8b`)	8B decoder-only	Strong function calling, multilingual (incl. Chinese), fits 16GB-class Radeon GPUs when quantized
Embedding	BAAI/bge-m3	1024-dim multilingual embedding	High-quality Chinese/English retrieval; used for both ingestion and query encoding
Reranker	BAAI/bge-reranker-v2-m3	cross-encoder (512 tokens)	Multilingual cross-encoder; substantially improves top-k precision over bi-encoder scores

`agent/parser.py`	Multi-format document parsing and smart chunking		
Backend	Runtime stack	Target environment	
Ollama	Ollama with automatic ROCm device backend on Linux (`ollama ps` confirms 100% GPU)	Simplest local setup; Windows/macOS/Linux development	
llama.cpp (HIP)	GGUF + `CMAKE_ARGS="-DGGML_HIP=ON"`, `n_gpu_layers=-1` full GPU offload	Single-machine, single-GPU, maximum control over quantization	
vLLM (ROCm)	`rocm/vllm` container; OpenAI-compatible API; high concurrency	Radeon Cloud and multi-tenant/high-concurrency GPU environments	
Quantization	Approx. VRAM	Speed	Recommendation
`Q8_0`	~8.2 GB	Fast	Quality-first deployments
`Q4_K_M`	~4.6 GB	Fastest	Balanced default recommendation
`agent/planner.py`	Task planning; `agent/memory.py` short- and long-term memory		
`ui/server.py`	FastAPI + SSE backend; document ingest; session management; GPU telemetry		
`benchmarks/bench_amd.py`	Automatic AMD/ROCm detection and tokens/s benchmark harness		
Role	Model	Size / Type	Rationale
LLM	Qwen3-8B (`qwen3:8b`)	8B decoder-only	Strong function calling, multilingual (incl. Chinese), fits 16GB-class Radeon GPUs when quantized
Embedding	BAAI/bge-m3	1024-dim multilingual embedding	High-quality Chinese/English retrieval; used for both ingestion and query encoding

`agent/planner.py`	Task planning; `agent/memory.py` short- and long-term memory		
Reranker	BAAI/bge-reranker-v2-m3	cross-encoder (512 tokens)	Multilingual cross-encoder; substantially improves top-k precision over bi-encoder scores
Backend	Runtime stack	Target environment	
Ollama	Ollama with automatic ROCm device backend on Linux (`ollama ps` confirms 100% GPU)	Simplest local setup; Windows/macOS/Linux development	
llama.cpp (HIP)	GGUF + `CMAKE_ARGS="-DGGML_HIP=ON"`, `n_gpu_layers=-1` full GPU offload	Single-machine, single-GPU, maximum control over quantization	
vLLM (ROCm)	`rocm/vllm` container; OpenAI-compatible API; high concurrency	Radeon Cloud and multi-tenant/high-concurrency GPU environments	
Quantization	Approx. VRAM	Speed	Recommendation
`Q8_0`	~8.2 GB	Fast	Quality-first deployments
`Q4_K_M`	~4.6 GB	Fastest	Balanced default recommendation
`ui/server.py`	FastAPI + SSE backend; document ingest; session management; GPU telemetry		
`benchmarks/bench_amd.py`	Automatic AMD/ROCm detection and tokens/s benchmark harness		
Role	Model	Size / Type	Rationale
LLM	Qwen3-8B (`qwen3:8b`)	8B decoder-only	Strong function calling, multilingual (incl. Chinese), fits 16GB-class Radeon GPUs when quantized

`ui/server.py`	FastAPI + SSE backend; document ingest; session management; GPU telemetry		
Embedding	BAAI/bge-m3	1024-dim multilingual embedding	High-quality Chinese/English retrieval; used for both ingestion and query encoding
Reranker	BAAI/bge-reranker-v2-m3	cross-encoder (512 tokens)	Multilingual cross-encoder; substantially improves top-k precision over bi-encoder scores
Backend	Runtime stack	Target environment	
Ollama	Ollama with automatic ROCm device backend on Linux (`ollama ps` confirms 100% GPU)	Simplest local setup; Windows/macOS/Linux development	
llama.cpp (HIP)	GGUF + `CMAKE_ARGS="-DGGML_HIP=ON"`, `n_gpu_layers=-1` full GPU offload	Single-machine, single-GPU, maximum control over quantization	
vLLM (ROCm)	`rocm/vllm` container; OpenAI-compatible API; high concurrency	Radeon Cloud and multi-tenant/high-concurrency GPU environments	
Quantization	Approx. VRAM	Speed	Recommendation
`Q8_0`	~8.2 GB	Fast	Quality-first deployments
`Q4_K_M`	~4.6 GB	Fastest	Balanced default recommendation
`benchmarks/bench_amd.py`	Automatic AMD/ROCm detection and tokens/s benchmark harness		
Role	Model	Size / Type	Rationale
LLM	Qwen3-8B (`qwen3:8b`)	8B decoder-only	Strong function calling, multilingual (incl. Chinese), fits 16GB-class Radeon GPUs when quantized

`benchmarks/bench_amd.py`	Automatic AMD/ROCm detection and tokens/s benchmark harness		
Embedding	BAAI/bge-m3	1024-dim multilingual embedding	High-quality Chinese/English retrieval; used for both ingestion and query encoding
Reranker	BAAI/bge-reranker-v2-m3	cross-encoder (512 tokens)	Multilingual cross-encoder; substantially improves top-k precision over bi-encoder scores
Backend	Runtime stack	Target environment	
Ollama	Ollama with automatic ROCm device backend on Linux (`ollama ps` confirms 100% GPU)	Simplest local setup; Windows/macOS/Linux development	
llama.cpp (HIP)	GGUF + `CMAKE_ARGS="-DGGML_HIP=ON"`, `n_gpu_layers=-1` full GPU offload	Single-machine, single-GPU, maximum control over quantization	
vLLM (ROCm)	`rocm/vllm` container; OpenAI-compatible API; high concurrency	Radeon Cloud and multi-tenant/high-concurrency GPU environments	
Quantization	Approx. VRAM	Speed	Recommendation
`Q8_0`	~8.2 GB	Fast	Quality-first deployments
`Q4_K_M`	~4.6 GB	Fastest	Balanced default recommendation

---

4. Core Capabilities

4.1 Offline Multi-Agent RAG

A full Decomposer → parallel Researchers → Fact-Checker → Synthesizer pipeline runs entirely offline. Complex questions are decomposed into independent sub-questions; researchers execute in parallel; a fact-checker verifies each sub-report; and the synthesizer produces a single integrated answer that cites sources in `[ :doc]` format.

4.2 Hybrid Retrieval with Reranking

Two retrieval signals are fused for recall and precision:

- Query rewriting — a lightweight LLM call rewrites the query into up to 3 retrieval

variants (retaining intent, completing proper nouns/acronyms), improving recall on paraphrase-heavy questions.

- Vector search — ChromaDB with `bge-m3` embeddings (1024-dim, cosine distance), tuned for multilingual and Chinese content.
- BM25 keyword search — exact-keyword recall that complements semantic search on proper nouns, abbreviations, and version identifiers (e.g. "Q4_K_M", "ROCm 6.x").
- Reciprocal Rank Fusion (RRF) — rank-based fusion (k=60) of both result lists, with a configurable BM25 weight (default 0.4) that is robust to score-calibration mismatch.
- Cross-encoder rerank — `BAAI/bge-reranker-v2-m3` jointly scores (query, passage) pairs and reorders the fused candidate pool (default 16 → top 4). Cross-encoder relevance is markedly more accurate than bi-encoder distance and is the single highest-leverage point for RAG precision.

The entire pipeline is local and offline; every stage degrades gracefully (pure vector search is the guaranteed fallback).

4.3 Verifiable, Citable Answers

Every answer sentence is checked against the retrieved source text (see Section 7). Citations are enforced to reference only documents that actually exist in the knowledge base. The UI exposes a per-sentence verification panel and a composite trust score.

4.4 Per-Sentence Groundedness Panel

The UI renders each answer sentence with its support level — supported / partial / unsupported — and its grounding score. Unsupported content is surfaced honestly to the user rather than silently presented as fact.

4.5 Multi-Format Document Import

Documents are parsed from PDF, Word (.docx), Markdown, Excel (.xlsx), CSV, JSON, PPT (.pptx) and plain text, chunked at 512 characters with a 64-character overlap, embedded, and stored in ChromaDB. Ingestion is idempotent (content-hash based); documents can be added and listed through the web UI or CLI (`python main.py ingest`), and removed through the web UI.

4.6 Live GPU Monitoring

A dedicated panel polls GPU utilization, VRAM usage, temperature, power draw and core clocks via `pyamdgpuinfo` or `rocm-smi --json` (with a demo-data fallback on non-AMD development machines), letting the user watch the parallelism and quantization strategy in action during inference.

4.7 Memory and Planning Tools

The agent supports multi-turn short-term memory (configurable, default last 10 turns) and persistent long-term memory, native function-calling tools (`rag_search`, `read_doc`, `list_docs`, `summarize`, `web_search_offline`), and a planner that auto-triggers on complex questions (compare/analyze/plan keywords) to keep single-shot answers fast.

---

5. Model Stack & Local Deployment Plan

5.1 Model Selection

Role	Model	Size / Type	Rationale
LLM	Qwen3-8B (`qwen3:8b`)	8B decoder-only	Strong function calling, multilingual (incl. Chinese), fits 16GB-class Radeon GPUs when quantized
Embedding	BAAI/bge-m3	1024-dim multilingual embedding	High-quality Chinese/English retrieval; used for both ingestion and query encoding
Reranker	BAAI/bge-reranker-v2-m3	cross-encoder (512 tokens)	Multilingual cross-encoder; substantially improves top-k precision over bi-encoder scores
Backend	Runtime stack	Target environment	
Ollama	Ollama with automatic ROCm device backend on Linux (`ollama ps` confirms 100% GPU)	Simplest local setup; Windows/macOS/Linux development	
llama.cpp (HIP)	GGUF + `CMAKE_ARGS="-DGGML_HIP=ON"`, `n_gpu_layers=-1` full GPU offload	Single-machine, single-GPU, maximum control over quantization	
vLLM (ROCm)	`rocm/vllm` container; OpenAI-compatible API; high concurrency	Radeon Cloud and multi-tenant/high-concurrency GPU environments	
Quantization	Approx. VRAM	Speed	Recommendation
`Q8_0`	~8.2 GB	Fast	Quality-first deployments
`Q4_K_M`	~4.6 GB	Fastest	Balanced default recommendation

LLM	Qwen3-8B (`qwen3:8b`)	8B decoder-only	Strong function calling, multilingual (incl. Chinese), fits 16GB-class Radeon GPUs when quantized
Embedding	BAAI/bge-m3	1024-dim multilingual embedding	High-quality Chinese/English retrieval; used for both ingestion and query encoding
Reranker	BAAI/bge-reranker-v2-m3	cross-encoder (512 tokens)	Multilingual cross-encoder; substantially improves top-k precision over bi-encoder scores
Backend	Runtime stack	Target environment	
Ollama	Ollama with automatic ROCm device backend on Linux (`ollama ps` confirms 100% GPU)	Simplest local setup; Windows/macOS/Linux development	
llama.cpp (HIP)	GGUF + `CMAKE_ARGS="-DGGML_HIP=ON"`, `n_gpu_layers=-1` full GPU offload	Single-machine, single-GPU, maximum control over quantization	
vLLM (ROCm)	`rocm/vllm` container; OpenAI-compatible API; high concurrency	Radeon Cloud and multi-tenant/high-concurrency GPU environments	
Quantization	Approx. VRAM	Speed	Recommendation
`Q8_0`	~8.2 GB	Fast	Quality-first deployments
`Q4_K_M`	~4.6 GB	Fastest	Balanced default recommendation

LLM	Qwen3-8B (`qwen3:8b`)	8B decoder-only	Strong function calling, multilingual (incl. Chinese), fits 16GB-class Radeon GPUs when quantized
Embedding	BAAI/bge-m3	1024-dim multilingual embedding	High-quality Chinese/English retrieval; used for both ingestion and query encoding
Reranker	BAAI/bge-reranker-v2-m3	cross-encoder (512 tokens)	Multilingual cross-encoder; substantially improves top-k precision over bi-encoder scores
Backend	Runtime stack	Target environment	
Ollama	Ollama with automatic ROCm device backend on Linux (`ollama ps` confirms 100% GPU)	Simplest local setup; Windows/macOS/Linux development	
llama.cpp (HIP)	GGUF + `CMAKE_ARGS="-DGGML_HIP=ON"`, `n_gpu_layers=-1` full GPU offload	Single-machine, single-GPU, maximum control over quantization	
vLLM (ROCm)	`rocm/vllm` container; OpenAI-compatible API; high concurrency	Radeon Cloud and multi-tenant/high-concurrency GPU environments	
Quantization	Approx. VRAM	Speed	Recommendation
`Q8_0`	~8.2 GB	Fast	Quality-first deployments
`Q4_K_M`	~4.6 GB	Fastest	Balanced default recommendation

Embedding	BAAI/bge-m3	1024-dim multilingual embedding	High-quality Chinese/English retrieval; used for both ingestion and query encoding
Reranker	BAAI/bge-reranker-v2-m3	cross-encoder (512 tokens)	Multilingual cross-encoder; substantially improves top-k precision over bi-encoder scores
Backend	Runtime stack	Target environment	
Ollama	Ollama with automatic ROCm device backend on Linux (`ollama ps` confirms 100% GPU)	Simplest local setup; Windows/macOS/Linux development	
llama.cpp (HIP)	GGUF + `CMAKE_ARGS="-DGGML_HIP=ON"`, `n_gpu_layers=-1` full GPU offload	Single-machine, single-GPU, maximum control over quantization	
vLLM (ROCm)	`rocm/vllm` container; OpenAI-compatible API; high concurrency	Radeon Cloud and multi-tenant/high-concurrency GPU environments	
Quantization	Approx. VRAM	Speed	Recommendation
`Q8_0`	~8.2 GB	Fast	Quality-first deployments
`Q4_K_M`	~4.6 GB	Fastest	Balanced default recommendation
Reranker	BAAI/bge-reranker-v2-m3	cross-encoder (512 tokens)	Multilingual cross-encoder; substantially improves top-k precision over bi-encoder scores
Backend	Runtime stack	Target environment	
Ollama	Ollama with automatic ROCm device backend on Linux (`ollama ps` confirms 100% GPU)	Simplest local setup; Windows/macOS/Linux development	

Reranker	BAAI/bge-reranker-v2-m3	cross-encoder (512 tokens)	Multilingual cross-encoder; substantially improves top-k precision over bi-encoder scores
llama.cpp (HIP)	GGUF + `CMAKE_ARGS="-DGGML_HIP=ON"`, `n_gpu_layers=-1` full GPU offload	Single-machine, single-GPU, maximum control over quantization	
vLLM (ROCm)	`rocm/vllm` container; OpenAI-compatible API; high concurrency	Radeon Cloud and multi-tenant/high-concurrency GPU environments	
Quantization	Approx. VRAM	Speed	Recommendation
`Q8_0`	~8.2 GB	Fast	Quality-first deployments
`Q4_K_M`	~4.6 GB	Fastest	Balanced default recommendation

The stack is deliberately small-model focused: a quantized 8B-class LLM, a 1024-dim embedding model, and a compact cross-encoder together fit comfortably in the 16–24GB VRAM class of AMD workstation GPUs, keeping cost and power low while retaining strong reasoning and retrieval quality.

5.2 Inference Backends

The `LLMBackend` abstraction exposes one `chat` / `chat_stream` / `embed` interface regardless of backend; a single `config.yaml` switch selects the runtime.

Backend	Runtime stack	Target environment	
Ollama	Ollama with automatic ROCm device backend on Linux (`ollama ps` confirms 100% GPU)	Simplest local setup; Windows/macOS/Linux development	
llama.cpp (HIP)	GGUF + `CMAKE_ARGS="-DGGML_HIP=ON"`, `n_gpu_layers=-1` full GPU offload	Single-machine, single-GPU, maximum control over quantization	
vLLM (ROCm)	`rocm/vllm` container; OpenAI-compatible API; high concurrency	Radeon Cloud and multi-tenant/high-concurrency GPU environments	
Quantization	Approx. VRAM	Speed	Recommendation
`Q8_0`	~8.2 GB	Fast	Quality-first deployments

Backend	Runtime stack	Target environment	
`Q4_K_M`	~4.6 GB	Fastest	Balanced default recommendation
Ollama	Ollama with automatic ROCm device backend on Linux (`ollama ps` confirms 100% GPU)	Simplest local setup; Windows/macOS/Linux development	
llama.cpp (HIP)	GGUF + `CMAKE_ARGS="-DGGML_HIP=ON"`, `n_gpu_layers=-1` full GPU offload	Single-machine, single-GPU, maximum control over quantization	
vLLM (ROCm)	`rocm/vllm` container; OpenAI-compatible API; high concurrency	Radeon Cloud and multi-tenant/high-concurrency GPU environments	
Quantization	Approx. VRAM	Speed	Recommendation
`Q8_0`	~8.2 GB	Fast	Quality-first deployments
`Q4_K_M`	~4.6 GB	Fastest	Balanced default recommendation
Ollama	Ollama with automatic ROCm device backend on Linux (`ollama ps` confirms 100% GPU)	Simplest local setup; Windows/macOS/Linux development	
llama.cpp (HIP)	GGUF + `CMAKE_ARGS="-DGGML_HIP=ON"`, `n_gpu_layers=-1` full GPU offload	Single-machine, single-GPU, maximum control over quantization	
vLLM (ROCm)	`rocm/vllm` container; OpenAI-compatible API; high concurrency	Radeon Cloud and multi-tenant/high-concurrency GPU environments	
Quantization	Approx. VRAM	Speed	Recommendation
`Q8_0`	~8.2 GB	Fast	Quality-first deployments
`Q4_K_M`	~4.6 GB	Fastest	Balanced default recommendation

llama.cpp (HIP)	GGUF + `CMAKE_ARGS="-DGGML_HIP=ON"`, `n_gpu_layers=-1` full GPU offload	Single-machine, single-GPU, maximum control over quantization	
vLLM (ROCm)	`rocm/vllm` container; OpenAI-compatible API; high concurrency	Radeon Cloud and multi-tenant/high-concurrency GPU environments	
Quantization	Approx. VRAM	Speed	Recommendation
`Q8_0`	~8.2 GB	Fast	Quality-first deployments
`Q4_K_M`	~4.6 GB	Fastest	Balanced default recommendation
vLLM (ROCm)	`rocm/vllm` container; OpenAI-compatible API; high concurrency	Radeon Cloud and multi-tenant/high-concurrency GPU environments	
Quantization	Approx. VRAM	Speed	Recommendation
`Q8_0`	~8.2 GB	Fast	Quality-first deployments
`Q4_K_M`	~4.6 GB	Fastest	Balanced default recommendation

5.3 Quantization

GGUF quantization balances VRAM footprint against speed and quality. Both variants are fully GPU-offloaded:

Quantization	Approx. VRAM	Speed	Recommendation
`Q8_0`	~8.2 GB	Fast	Quality-first deployments
`Q4_K_M`	~4.6 GB	Fastest	Balanced default recommendation
LLM	Qwen3-8B (`qwen3:8b`)	8B decoder-only	Strong function calling, multilingual (incl. Chinese), fits 16GB-class Radeon GPUs when quantized
Embedding	BAAI/bge-m3	1024-dim multilingual embedding	High-quality Chinese/English retrieval; used for both ingestion and query encoding

LLM	Qwen3-8B (`qwen3:8b`)	8B decoder-only	Strong function calling, multilingual (incl. Chinese), fits 16GB-class Radeon GPUs when quantized
Reranker	BAAI/bge-reranker-v2-m3	cross-encoder (512 tokens)	Multilingual cross-encoder; substantially improves top-k precision over bi-encoder scores
Backend	Runtime stack	Target environment	
Ollama	Ollama with automatic ROCm device backend on Linux (`ollama ps` confirms 100% GPU)	Simplest local setup; Windows/macOS/Linux development	
llama.cpp (HIP)	GGUF + `CMAKE_ARGS="-DGGML_HIP=ON"`, `n_gpu_layers=-1` full GPU offload	Single-machine, single-GPU, maximum control over quantization	
vLLM (ROCm)	`rocm/vllm` container; OpenAI-compatible API; high concurrency	Radeon Cloud and multi-tenant/high-concurrency GPU environments	
Quantization	Approx. VRAM	Speed	Recommendation
`Q8_0`	~8.2 GB	Fast	Quality-first deployments
`Q4_K_M`	~4.6 GB	Fastest	Balanced default recommendation
`Q8_0`	~8.2 GB	Fast	Quality-first deployments
`Q4_K_M`	~4.6 GB	Fastest	Balanced default recommendation
`Q4_K_M`	~4.6 GB	Fastest	Balanced default recommendation

`Q4_K_M` is the recommended default: it fits comfortably alongside the embedding and reranker models within 16GB VRAM, leaving headroom for long context and concurrent researcher requests.

5.4 Deployment Paths

Path A — Local (pip + Ollama). Install Python 3.10+, pull `qwen3:8b` and `bge-m3` into Ollama, `pip install -r requirements.txt`, then:

```
docker compose up -d --build
```

```
docker exec -it private-rag-ollama ollama pull qwen3:8b
docker exec -it private-rag-ollama ollama pull bge-m3
```

Path B — Docker Compose. The repository ships a `docker-compose.yml` running the application plus an Ollama container. On AMD + ROCm hosts, uncommenting the `devices` section in the compose file passes the GPU into the container:

```
rocminfo                  # confirm ROCm device presence
rocm-smi                  # live GPU memory / temperature / utilization
ollama ps                 # confirm model is 100% GPU-resident
python benchmarks/bench_amd.py   # auto-detect and output tokens/s
```

Path C — AMD Radeon Cloud (`deploy_rc.sh`). A one-shot script for Radeon Cloud (Jupyter workspace, ROCm + PyTorch preinstalled): detects the AMD GPU via `amd-smi`/`rocm-smi`; starts a vLLM (ROCm) inference server (auto-detecting RC-preloaded models such as `Qwen3.6-35B-A3B-AWQ-4bit`, else `Qwen/Qwen3-8B`); installs dependencies; switches `config.yaml` to the vLLM backend with `sentence-transformers` embeddings (keeping `bge-m3` at 1024-dim for vector-database compatibility); and runs an end-to-end inference smoke test.

5.5 Hardware Requirements

Resource	Minimum	Recommended
GPU	AMD Radeon with ROCm support, 16GB VRAM	Radeon RX 7900 XTX / W7900 (or Radeon Cloud instance)
ROCm	ROCm 6.x (Linux)	ROCm 6.x + Ollama/llama.cpp ROCm build
RAM	16 GB	32 GB
Python	3.10+	3.11
GPU	AMD Radeon with ROCm support, 16GB VRAM	Radeon RX 7900 XTX / W7900 (or Radeon Cloud instance)
ROCm	ROCm 6.x (Linux)	ROCm 6.x + Ollama/llama.cpp ROCm build
RAM	16 GB	32 GB
Python	3.10+	3.11
GPU	AMD Radeon with ROCm support, 16GB VRAM	Radeon RX 7900 XTX / W7900 (or Radeon Cloud instance)
ROCm	ROCm 6.x (Linux)	ROCm 6.x + Ollama/llama.cpp ROCm build
RAM	16 GB	32 GB
Python	3.10+	3.11

ROCm	ROCm 6.x (Linux)	ROCm 6.x + Ollama/llama.cpp ROCm build
RAM	16 GB	32 GB
Python	3.10+	3.11
RAM	16 GB	32 GB
Python	3.10+	3.11
Python	3.10+	3.11

---

6. AMD Radeon GPU / ROCm Optimization

This section is the direct response to the Track 2 scoring emphasis (AMD Radeon GPU / ROCm optimization, 40%).

6.1 Native ROCm Inference Backend

The default inference stack is ROCm-native: llama.cpp compiled with `GGML_HIP=ON`, or Ollama's ROCm backend, or vLLM on ROCm wheels for Radeon Cloud. No CUDA-specific path is required, and model weights run fully on the Radeon GPU (`n_gpu_layers=-1`, `ollama ps` shows 100% GPU residency).

6.2 GGUF Quantization

Quantization reduces both VRAM footprint and memory bandwidth per token. `Q4_K_M` (~4.6GB) is the recommended operating point for an 8B model on 16GB GPUs, with `Q8_0` (~8.2GB) available when maximum precision is needed. Because Radeon VRAM bandwidth is the dominant throughput constraint for LLM decode, smaller quantizations translate almost directly into higher tokens/s.

6.3 Module-Level Model Singletons (VRAM Budget)

Embedding and cross-encoder reranker models are loaded as module-level singletons and shared across all agents and RAGStore instances. In multi-agent mode, `n` researchers therefore add only inference concurrency, not `n` extra copies of the embedding and reranker in VRAM. Models are also lazily loaded on first use, so the service does not pre-allocate VRAM at startup.

6.4 Multi-Agent Parallelism to Saturate the GPU

The orchestrator runs `n` researcher agents concurrently via a thread pool. On a single local GPU, concurrent decode requests are interleaved to keep compute units busy; this turns the "serialized multi-step research" latency into the latency of the slowest single sub-task. The wall-clock gain grows with sub-question count, and the shared model singletons ensure the gain is not paid for in VRAM.

6.5 Cross-Encoder Rerank on GPU

Reranking runs the cross-encoder through the same ROCm path, keeping rerank latency to millisecond scale while adding the precision that dominates final answer quality.

6.6 Concurrency Safety on Native Libraries (Engineering Note)

ChromaDB's underlying hnswlib and PyTorch's cross-encoder are not thread-safe under concurrent `collection.query` / `collection.get` / reranker inference, which can produce intermittent segfaults (exit code 139) under multi-agent retrieval. All paths touching these native resources — `search()`, corpus loading, document add/remove — are therefore serialized through a process-level reentrant lock (`threading.RLock`). The RLock is required because `search()` internally re-enters corpus loading; LLM inference (the dominant cost) remains naturally concurrent through the Ollama/vLLM service, so the lock adds only milliseconds of serialization while eliminating the crash class.

6.7 Verification and Monitoring Tooling

```
python benchmarks/bench_amd.py --repeat 3
```

---

7. Rigorousness and Trustworthiness

The innovation highlight of this project. LLM hallucination is the central trust problem for private assistants. This system makes each claim machine-auditable through a three-layer verification stack, all local and mostly deterministic.

7.1 Groundedness Verification (Deterministic)

For every sentence of an answer, the verifier computes character-level 3-gram overlap between the normalized sentence and the normalized retrieved source passages (stop-words and punctuation removed; 3-gram is robust to light paraphrasing). Each sentence is labeled:

- supported — overlap ratio > 0.30
- partial — overlap ratio > 0.12
- unsupported — otherwise

The overall grounding score is the mean support ratio across sentences. This is fully deterministic (no LLM), fast, and explainable — the exact overlap that produced each label can be displayed. Because sub-reports are verified against the full document text of the actually retrieved sources (not truncated tool outputs), scores are accurate even when the final answer is lightly rephrased.

7.2 Citation Verification

Answers are required to cite sources in `[ :filename]` form. The verifier:

· Parses citations from the answer via regex.

· Cross-checks each cited filename against the set of documents actually retrieved in this session.

· Reports citation precision (fraction of citations that exist in the knowledge base), and flags every invalid citation.

If the answer cites nothing, this is surfaced as "no sources cited" rather than silently passed — the user always knows the provenance status.

7.3 LLM-as-Judge (Fact-Check in Multi-Agent Mode)

In multi-agent mode, a third stage runs an LLM judge against each sub-report: the judge reads the sub-report plus its retrieved sources and emits a JSON array of per-claim verdicts (`{claim, supported, reason}`). Claims judged unsupported are collected and reported as flags (e.g., "verification found N unsupported claims: ..."). This is a best-effort stage — failures degrade gracefully without blocking the pipeline — but when it runs it adds semantic judgment on top of

the n-gram overlap.

7.4 Verify-Then-Synthesize (What the User Sees)

The synthesizer is instructed to compose the final answer only from sub-reports that passed verification, to keep `[ :doc]` citations to documents that truly exist, and to explicitly state when part of a sub-report is only partially or not supported. After synthesis, a final check re-runs groundedness and citation verification on the complete answer. The UI then shows:

- a composite trust score (grounding + citation precision);
- a per-sentence panel with supported/partial/unsupported labels and scores;
- a source deep-dive drawer — clicking any citation opens the original passage with

the query terms highlighted.

---

8. Performance Benchmark Plan

The repository ships `benchmarks/bench_amd.py`, which auto-detects the AMD/ROCm environment (`rocminfo` / `rocm-smi`), warms the model, and measures tokens/s and latency over repeat runs for a standard prompt set.

8.1 Baseline (Development Machine)

Measured on the development machine (NVIDIA RTX 4070 Laptop, Ollama `qwen3:8b`) with the benchmark harness:

tric	Value			
U inference oughput	~1.8 tokens/s			
nfig	Tokens/s (measured)	First-token latency	VRAM	Notes
U (baseline, dev)	~1.8	—	—	reference point
7900 / ROCm, GGUF _K_M, full offload	[fill in]	[fill in]	~4.6 GB	balanced default
7900 / ROCm, GGUF _0, full offload	[fill in]	[fill in]	~8.2 GB	quality mode
deon Cloud, vLLM	[fill in]	[fill in]	—	high concurrency
U inference oughput	~1.8 tokens/s			
nfig	Tokens/s (measured)	First-token latency	VRAM	Notes
U (baseline, dev)	~1.8	—	—	reference point
7900 / ROCm, GGUF _K_M, full offload	[fill in]	[fill in]	~4.6 GB	balanced default
7900 / ROCm, GGUF _0, full offload	[fill in]	[fill in]	~8.2 GB	quality mode
deon Cloud, vLLM	[fill in]	[fill in]	—	high concurrency

U inference oughput	~1.8 tokens/s			
nfig	Tokens/s (measured)	First-token latency	VRAM	Notes
U (baseline, dev)	~1.8	—	—	reference point
7900 / ROCm, GGUF _K_M, full offload	[fill in]	[fill in]	~4.6 GB	balanced default
7900 / ROCm, GGUF _0, full offload	[fill in]	[fill in]	~8.2 GB	quality mode
deon Cloud, vLLM	[fill in]	[fill in]	—	high concurrency

8.2 Expected Results on AMD Radeon GPU / ROCm

The table below is the committed benchmark target, to be filled with measured results on Radeon Cloud (RX 7900-class / ROCm) ahead of submission. Expected speedup: 5–10x over the CPU baseline.

Config	Tokens/s (measured)	First-token latency	VRAM	Notes
CPU (baseline, dev)	~1.8	—	—	reference point
RX 7900 / ROCm, GGUF Q4_K_M, full offload	[fill in]	[fill in]	~4.6 GB	balanced default
RX 7900 / ROCm, GGUF Q8_0, full offload	[fill in]	[fill in]	~8.2 GB	quality mode
Radeon Cloud, vLLM	[fill in]	[fill in]	—	high concurrency
CPU (baseline, dev)	~1.8	—	—	reference point
RX 7900 / ROCm, GGUF Q4_K_M, full offload	[fill in]	[fill in]	~4.6 GB	balanced default
RX 7900 / ROCm, GGUF Q8_0, full offload	[fill in]	[fill in]	~8.2 GB	quality mode
Radeon Cloud, vLLM	[fill in]	[fill in]	—	high concurrency

CPU (baseline, dev)	~1.8	—	—	reference point
RX 7900 / ROCm, GGUF Q4_K_M, full offload	[fill in]	[fill in]	~4.6 GB	balanced default
RX 7900 / ROCm, GGUF Q8_0, full offload	[fill in]	[fill in]	~8.2 GB	quality mode
Radeon Cloud, vLLM	[fill in]	[fill in]	—	high concurrency
RX 7900 / ROCm, GGUF Q4_K_M, full offload	[fill in]	[fill in]	~4.6 GB	balanced default
RX 7900 / ROCm, GGUF Q8_0, full offload	[fill in]	[fill in]	~8.2 GB	quality mode
Radeon Cloud, vLLM	[fill in]	[fill in]	—	high concurrency
RX 7900 / ROCm, GGUF Q8_0, full offload	[fill in]	[fill in]	~8.2 GB	quality mode
Radeon Cloud, vLLM	[fill in]	[fill in]	—	high concurrency
Radeon Cloud, vLLM	[fill in]	[fill in]	—	high concurrency

Command to reproduce:

```
private-rag-agent/
├── agent/
│   ├── agent.py        # Agent main loop (planning / tools / memory / citation check)
│   ├── multi_agent.py  # Multi-agent orchestration (decompose→research→check→synthesize)
│   ├── rag.py          # Hybrid retrieval (vector   BM25 → RRF → rerank)
│   ├── verify.py       # Groundedness + citation verification
│   ├── llm.py          # Unified inference backends (ollama / llama.cpp / vLLM)
│   ├── parser.py       # Multi-format document parsing + smart chunking
│   ├── tools.py        # Tool registry (function calling)
│   ├── memory.py       # Short-term + long-term memory
│   └── planner.py      # Task planning
├── ui/
│   ├── server.py       # FastAPI + SSE streaming endpoints
│   └── static/         # Native HTML/CSS/JS professional dark UI
├── benchmarks/
│   └── bench_amd.py    # AMD ROCm benchmark harness
├── data/docs/          # Knowledge base (6 markdown documents, 15 chunks)
├── config.yaml         # Model / retrieval / chunking / quantization configuration
├── main.py             # Entry point (ingest / ask / ui)
├── deploy_rc.sh        # Radeon Cloud one-shot deployment script
├── docker-compose.yml  # Containerized deployment
├── docs/               # This specification + submission materials
```

```
└── requirements.txt
```

9. Repository Structure

10. Roadmap

- Final AMD benchmark pass — fill Section 8.2 with measured RX 7900-class / Radeon Cloud numbers, including multi-agent parallel speedup curves.
- Agent memory upgrades — richer long-term memory consolidation and cross-session personalization.
- Knowledge-base scale-out — sharded vector collections for corpora beyond the single-host default.
- Structured evaluation — RAGAS-style faithfulness/answer-relevancy scores on a curated gold set, to complement the deterministic n-gram verification.
- Multi-GPU (MIG-style) scheduling for Radeon Cloud deployments with multiple accelerators.